BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION INFORMATICĂ ENGLEZĂ

DIPLOMA THESIS

Doclane

Supervisor
Lect. PETRESCU Manuela

Author
Robert Béres

2026

ABSTRACT

This thesis presents Doclane, a web-based system designed to improve document-based workflows in public institutions. In many countries, including Romania, public services still depend on paper documents and physical visits. Citizens often need to go to institutions in person, wait in queues, and submit printed forms. This process takes a lot of time and can be inconvenient.

The main problem is that current systems are slow and not fully digital. Even when some digital services exist, they are often limited to one institution or only cover part of the process. For example, some systems only allow document submission or signing, but not the full workflow from submission to approval and feedback.

Doclane is designed to solve this problem by creating a simple digital workflow between citizens and public institutions. Users can submit requests online, upload required documents, and track the status of their application. Institution employees can review submissions, validate documents, and give feedback when something is missing or incorrect.

The system is built as a web application. The backend is written in Go, and the frontend uses Next.js. Authentication is handled using JWT tokens, and files are stored using cloud storage. The system is structured in layers to keep the code clean and easy to maintain.

The goal of Doclane is to reduce the need for physical visits and paper documents. It also aims to make the process clearer for users by showing each step of the workflow. Instead of repeating visits and waiting for long periods, users can handle most of the process online.

This thesis also looks at existing systems such as GOV.UK, Enter Finland, and DocuSign. These systems solve parts of the problem, but they are either limited to one type of service or do not cover the full workflow. Doclane tries to combine these ideas into one simple system that can be used across different public institutions.

In conclusion, Doclane shows how digital tools can improve public administration by making document handling faster, more organized, and easier for citizens.

Contents

Chapter 1

Introduction

1.1 Introducing the problem

Public institutions play an important role in the daily lives of citizens, providing access to services such as official documents, permits, and administrative procedures. Because of this, it is important that the processes involved are efficient, reliable, and easy to use.

In many cases, these processes are still based on physical documents and in-person interactions. Citizens are often required to visit institutions, wait in queues, and submit printed files in order to complete simple requests. This approach leads to long waiting times, repeated visits, and inefficient handling of information.

In addition, physical documents can be lost, damaged, or incorrectly processed, which can delay important procedures. These inefficiencies do not only create inconvenience, but can also impact the ability of citizens to complete time-sensitive administrative tasks.

As a result, improving the way public institutions handle document-based workflows becomes an important problem to address.

1.2 Role of Information Technology in Public Administration

The use of information technology in public institutions has become increasingly important in recent years. Digital systems allow institutions to manage large amounts of data, automate processes, and reduce the time required to complete administrative tasks.

By moving services to digital platforms, interactions between citizens and institutions can be simplified. Instead of relying on physical documents and in-person

visits, many processes can be handled online, improving accessibility and reducing waiting times.

In addition, digital systems make it easier to store, organize, and retrieve information. This can improve decision-making and reduce errors caused by manual processing of documents.

However, the adoption of such systems is not always straightforward. Public institutions may face challenges such as limited financial resources, lack of technical infrastructure, and insufficient digital skills among both employees and citizens.

Despite these challenges, the use of information technology remains a key factor in improving the efficiency and quality of public services.

1.3 Challenges in Traditional Systems

Traditional document handling in public institutions comes with several practical challenges that affect both citizens and employees.

One of the main issues is the need for physical presence. In most cases, citizens are required to visit an institution in person in order to submit or collect documents. This creates queues and waiting times, especially during busy hours or peak periods.

Another problem is the use of paper-based documents. Files are printed, copied, and stored physically. This takes space and also creates the risk of documents being lost, damaged, or misplaced. It also makes it harder to search and manage information when needed.

The process of checking and validating documents is often manual. Employees need to go through each file one by one, which takes time and can lead to delays, especially when the number of requests is high.

Feedback to citizens is also not always immediate. If a document is missing or incorrect, the citizen usually finds out later, sometimes after waiting for a long time or making another visit. This leads to repeated submissions and additional waiting time.

In addition, many of these processes are not transparent to the user. Citizens often do not know the current status of their request or how long it will take until it is completed.

These challenges show that the traditional way of handling document-based workflows is slow and can be improved with more structured and digital solutions.

1.4 Limitations of existing digital solutions

Even though some public services already use digital platforms, they are often limited in scope. Most of these systems are built for a specific type of service or a single institution.

In many cases, users still need to switch between different systems depending on the service they need. This creates fragmentation and makes the overall process less consistent.

Some systems also only cover part of the workflow, such as submitting documents or signing them, but not the full process from submission to final approval and feedback.

Because of this, there is still a need for systems that can be reused across different institutions, where each institution can use its own instance of the same workflow structure instead of relying on completely separate tools.

1.5 Proposed solution

The proposed system, Doclane, is a web-based application designed to handle document-based workflows between citizens and public institutions.

The system allows users to submit requests, upload required documents, and track the status of their applications. On the other side, institution employees can review submissions, validate documents, and provide feedback when needed.

The goal of the system is to reduce the need for physical visits and repeated submissions by moving the process into a structured digital workflow.

Doclane also focuses on making the process more clear for users by showing what is required at each step and giving feedback when something is missing or incorrect.

1.6 Objectives

The main objective of this thesis is to design and implement a system that improves document-based workflows in public institutions.

The specific objectives include:

- Designing a backend system that can handle document requests.
- Implementing secure authentication and role-based access.
- Building a structured workflow for submission and validation.
- Using cloud storage for document handling.

- Making the system scalable and easy to extend.

1.7 Scope of the system

The system focuses on improving the workflow between citizens and public institutions when it comes to document-based requests.

It does not change the legal process of document approval. Final decisions are still made by authorized personnel within the institution.

The system is designed as a support tool that helps organize and digitalize the existing workflow, not replace institutional rules or procedures.

Chapter 2

Related work

This chapter presents existing systems and research related to digital document handling and public administration workflows. The goal is to show how similar problems have been handled in other systems and what solutions already exist.

The systems and studies presented here focus on document submission, electronic workflows, and online public services. Some of them are real systems used by governments, while others are research papers that propose ways to improve administrative processes.

After presenting these systems, a short comparison is made to identify their limitations in relation to the problem addressed by this thesis. This helps explain the motivation behind Doclane and how it differs from existing solutions.

2.1 Automated electronic document management systems in municipal units

Koptyakova et al. [KZM19] present a system for automated electronic document management in municipal institutions. The paper focuses on improving how public institutions handle documents and reducing the time needed for administrative work.

The authors explain that traditional document workflows in municipal units are slow and depend on manual work. Documents are usually registered, tracked, and processed by hand, which can cause delays when the number of requests is high.

To solve this, the paper proposes an automated system that digitizes document registration, tracking, and processing inside municipal institutions. This reduces manual work and makes the process faster.

The study also looks at the cost and benefits of such systems. It mentions that the payback period can be short because less manual work is needed and efficiency increases.

However, the system is mainly used inside one municipal institution. It focuses on internal document handling and not on the full process between citizens and institutions.

Compared to this, Doclane focuses on the full workflow between citizens and public institutions, including submission, validation, and feedback.

2.2 Digital document systems in public sector implementation

Az Ab Aziz et al. [AYMJ19] study the implementation of a Digital Document Management System (DDMS) in the Malaysian public sector. The system is used to manage documents and records inside government institutions and aims to reduce paper use and improve document handling.

The paper shows that even when such systems exist, adoption is not always high. The main issues are not only technical, but also related to policy, training, security, and internal support.

The authors explain that successful implementation depends on clear guidelines and coordination between agencies. Without this, different institutions use the system in different ways, which reduces its effectiveness.

Compared to this, Doclane focuses more on workflows between citizens and institutions, but the same idea applies: a system is only useful if it is properly adopted and used in real work.

2.3 Document Management Systems in Public Sector Integration

Electronic document management systems in public sector institutions are often part of larger information systems. International Journal of Advances in Engineering & Technology [dms12] explains that government institutions use many systems such as finance systems, HR systems, CRM systems, and web portals. Because of this, document management systems need to work with other systems.

The authors explain that integration means systems can exchange data and work together, not that they are merged into one system. This is important because many public sector processes depend on data moving between systems.

The study also shows that integration can improve efficiency by reducing manual work and making workflows simpler. However, it is not easy. It can be expensive, take time, and may require changes in several systems. Some integration

projects also fail due to poor planning.

Even with these problems, integration is still needed if institutions want to improve digital workflows.

Compared to this, Doclane also follows the idea of integration by connecting document workflows with different parts of an administrative system and improving communication between users and institutions.

2.4 Enter Finland

Enter Finland (`https://enterfinland.fi`) is a government service used in Finland for immigration applications. Users can fill forms online, upload documents, and pay fees. After submission, some cases still require an in-person visit.

The system reduces the need for paper submission and physical visits at the start of the process. However, it is limited to immigration services and one type of institution.

Compared to this, Doclane is designed for different types of document workflows across multiple institutions.

2.5 DocuSign

DocuSign (`https://www.docusign.com`) is a platform used for signing documents online. Users can upload documents, send them for signature, and track their status.

It removes the need to print and scan documents in many cases. It is mostly used in business environments.

However, it focuses mainly on signing and approval. It does not support full administrative workflows like structured requests or multi-step validation.

Compared to this, Doclane handles full workflows from submission to approval and feedback.

2.6 GOV.UK Services

GOV.UK (`https://www.gov.uk`) is a UK government platform for online public services. Users can apply for documents, submit forms, and access different services.

It reduces the need for physical visits and allows many tasks to be done online. It covers many services in one platform.

However, each service has its own workflow and is not part of one unified system.

Compared to this, Doclane aims to provide a single workflow system for different types of document requests.

2.7 Comparison

The systems described above solve parts of the same problem in different ways.

Enter Finland is focused on one domain. GOV.UK provides many services, but they are separate. DocuSign focuses on document signing.

All of them reduce paper use and improve digital access, but they are limited either by domain, structure, or function.

2.8 Gap

There is no widely used system that handles document-based workflows across multiple public institutions in one unified system.

Most systems are either domain-specific, split into separate services, or focused on one function like signing.

Doclane addresses this by providing a unified workflow for submission, validation, feedback, and approval across institutions.

Chapter 3

Implementation

The preferred programming language chosen for the backend was Golang, and the frontend was done in Next.js. Golang was chosen because it's a modern language, and although it is not object-oriented, the tradeoff is worth it for how fast it is and how rich of a standard library it has. It is also surprisingly fast to write, and one of the best things about it is that it has error objects. No more uncaught exceptions!

Next.js was chosen for its server side capabilities. Doclane is a dynamic app, it contains many HTTP calls and fetching everything client side would shortly become a nightmare when thinking of user experience.

Figure 3.1: Overview of Doclane's folder structure.

3.1 Backend

3.1.1 Golang

Go is a high-level, general-purpose programming language that is statically typed and compiled. It is known for the simplicity of its syntax and the efficiency of development that it enables through the inclusion of a large standard library supplying many needs for common projects.

3.1.2 Layers

Code is split into multiple layers:

- Handler layer: HTTP calls.
- Service layer: business logic and validation.
- Repository layer: interaction with the database.
- Model layer: structures used throughout the app.

Handler layer

The handler layer is responsible for extracting needed information from a request in order for it to be processed. Required information includes, but it is not limited to: JWT claims from the user, id of a resource that the user wishes to interact with, files that need to be uploaded.

Service layer

The service layer is responsible for validating a request passed from the handlers. Validations include, but are not limited to: checking the user's role, checking that they have sufficient permissions to take an action. One example would be: you cannot view the files from a request unless you are either an admin or you belong to the department managing that request.

Repository layer

The repository layer is responsible for executing queries on the database. It trusts everything it is given as everything given to it passes through the service first. No validations are done in the repository, ensuring a clean separation of concerns. Transactions, although rarely, are also used where multiple database operations are required for one operation to be considered complete in the app.

Model layer

The model layer simply holds the structures used throughout the app: users, requests, templates, invitation codes, comments, and more. Occasionally, DTOs (Data Transfer Objects) are used to enrich data coming from the database. A sample use case where a DTO is used is for retrieving requests, and making sure we also get the requesters' name and email, not just their ID, as the original model has only the ID in it.

3.1.3 Authentication

Authentication is done via JWTs (JSON Web Tokens). A JWT is a stateless form of authentication, it is basically a cryptographically signed payload containing the user id, the audience for which the token is intended, the issuer, but in Doclane's case it also contains the user's role, their department, their first and last names.

The information in the token is visible to anyone, it is not encrypted. However it cannot be altered (or one should say it can, but with no effect), because token verification will see that the signature does not match, therefore rejecting the token.

The token is stored in an HTTP only cookie, making it inaccessible to JavaScript running in the browser, which protects against attacks such as XSS (CrossSite Scripting).

```
6
7   type JWTClaims struct {
8           UserID       int    `json:"user_id"`
9           Email        string `json:"email"`
10          FirstName    string `json:"first_name"`
11          LastName     string `json:"last_name"`
12          Role         string `json:"role"`
13          DepartmentID *int   `json:"department_id,omitempty"`
14          jwt.StandardClaims
15  }
```

Figure 3.2: JWT Claims.

3.1.4 API Response

The entire API returns a common object structure across all responses, ensuring parsing a response from the callers' end a consistent process. Every request will come back with an object containing:

- "success" - a boolean that determines whether the request executed successfully.
- "message" - optional, a message, containing a short description of what happened.
- "error" - optional, an error message in case success boolean is false.
- "data" - optional, this is where any data is returned.

```
2
3   type APIResponse struct {
4           Success bool   `json:"success"`
5           Data    any    `json:"data,omitempty"`
6           Msg     string `json:"message,omitempty"`
7           Err     string `json:"error,omitempty"`
8   }
9
```

Figure 3.3: API Response struct.

3.1.5 Custom errors

Custom error objects have been defined to be used throughout the app. Reason for this is to have a unified way of treating them, and making code more readable.

3.1.6 Configuration

The single source of truth for what structures are being used throughout the app (i.e., which implementation of a repository is chosen, but not only) is the configuration file. It is from here that all other files use the service layer, the instances are being globally exported from this file.

This creates a scalable environment due to the fact that whenever a repository change is required, for example to use another interface implementation instead of the current one, all one has to do is come in this file and swap out that implementation. Simply put, just change one line of code and you're done.

3.1.7 Middleware

Middleware pattern is used all throughout the app. Almost every request is tested against having a valid JWT token, which has a signature that corresponds to an authentic token signed by the server.

This enables a significant amount of functions to be more lightweight, as they do not need to worry about checking whether a user has a valid token or not, but rather any request containing an invalid request is rejected before ever entering the handler layer.

There are two middleware checks happening in Doclane: first, a request is validated against having an authentic JWT signed by Doclane, and second it checks whether an account is active or not.

3.1.8 File storage

All the files that users upload within the app need to be stored somewhere. The storage chosen for these was S3 from AWS.

Without going into detail about S3 (Simple Storage Service), let's see the calls to the S3 API. S3 as a service is covered in the deployment chapter.

The file storage logic has been abstracted away as to not depend on a particular service. This means that the file storage logic is separate from application logic. It also lives in the service layer, since it's dealing with business logic.

All file access is done via S3 Presigned URLs, which is a feature of S3 in AWS that enables temporary access to a specific file, ensuring least privilege principle. More

on this in the deployment section.

3.1.9 Routing

Backend routing for requests is done using the third party Chi library. Although Golang's standard library is rich and has solutions for many things, HTTP routing is where it could be improved.

Chi helped in cleanly organizing the routes, almost in a tree-like manner. It's not a burden to maintain since the code is easy to read.

```
64          r.Get("/api/invitations/info", invitation_handler.GetInvitationCodeInfoHandler)
65          r.Post("/api/internal/process-recurring", request_handler.ProcessRecurringRequestsHandler)
66
67          r.Route("/api", func(r chi.Router) {
68                  r.Use(auth_middleware.AuthGuard)
69                  r.Use(auth_middleware.MustBeActive)
70
71                  r.Get("/stats", stats_handler.GetStatsHandler)
72
73                  r.Route("/users", func(r chi.Router) {
74                          r.Get("/", user_handler.GetUsersHandler)
75                          r.Get("/me", user_handler.GetCurrentUserHandler)
76                          r.Patch("/me/profile", user_handler.UpdateProfileHandler)
77                          r.Patch("/me/password", user_handler.UpdatePasswordHandler)
78                          r.Get("/by-department", user_handler.GetUsersByDepartmentHandler)
79                          r.Get("/{id}", user_handler.GetUserByIDHandler)
80                          r.Post("/notify/{id}", user_handler.NotifyUserHandler)
81                          r.Post("/deactivate/{id}", user_handler.DeactivateUserHandler)
82                          r.Patch("/{id}/department", user_handler.UpdateUserDepartmentHandler)
83                  })
84
85                  r.Route("/requests", func(r chi.Router) {
86                          r.Get("/", request_handler.GetAllRequestsHandler)
87                          r.Post("/", request_handler.AddRequestHandler)
88                          r.Get("/assignee/{id}", request_handler.GetRequestsByAssigneeHandler)
89                          r.Get("/department/{id}", request_handler.GetRequestsByDepartmentHandler)
```

Figure 3.4: Routing for the backend.

3.2 Database

PostgreSQL was the preferred choice for the project, mainly because it is one of the most popular engines out there, and a relational schema was needed.

Development has been done locally on a self-hosted PostgreSQL server, while the final app uses RDS from AWS, but more on this in the deployment chapter.

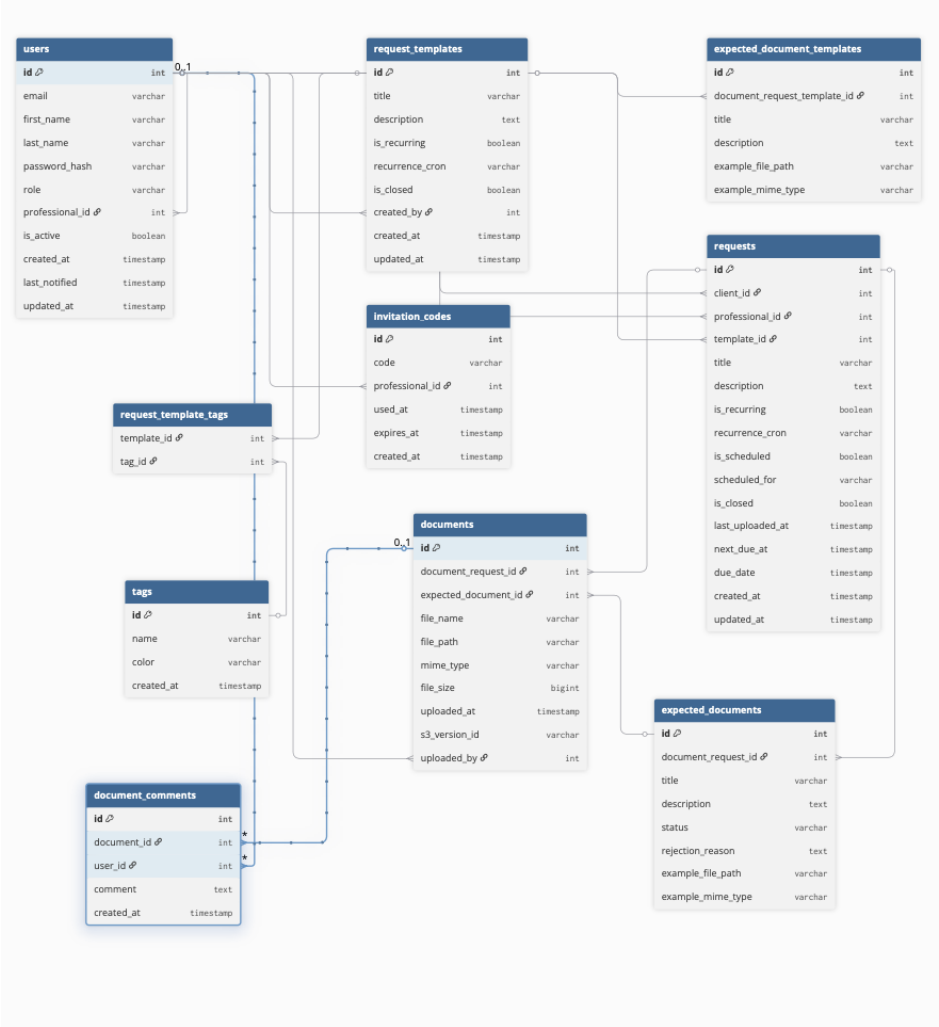

Figure 3.5: Database schema overview.

3.3 Frontend

Next.js is an open-source full-stack web development framework created by the private company Vercel providing React-based web applications with server-side rendering and static rendering.

Next.js was the preferred choice for the project due to its server-side capabilities. Doclane is inherently a dynamic app, fetching plenty of resources from the backend on right about every operation, therefore picking something like plain React.js would have not been optimal. If React.js was picked for example, the user would be waiting long times in loading screens, just fetching stuff. Meanwhile, Next.js pre-fetches data that the user needs with the help of its server-side capabilities and therefore loading times are faster.

Frontend code is split into small reusable components, such that the DRY (Don't Repeat Yourself) principle is followed. This is not only in the frontend, DRY is also used everywhere in the backend but truthfully in this part of the project it is the

most visible.

Chapter 4

Deployment

When it comes to deploying an application, there are many paths one could take. One might be ordering a physical server at home and running your application on there, another one could be running it locally on your machine where only people in your network can access it, and another one could be deploying the application in the cloud. The last one was chosen for Doclane.

4.1 Cloud computing

Cloud computing was preferred since it is the more modern approach, and reduces overhead with managing physical servers. Without the burden and expenses of buying, setting up and maintaining servers, more effort could be redirected to the development of the app and adding features that bring more value to it. It is also that renting those servers means you can just get rid of them whenever you're not using them anymore, and you only pay for how much you use them.

4.2 Amazon Web Services

The deployment of the app was done on AWS. The architecture is leveraging a mostly serverless architecture for a multitude of reasons, some of them being: cost efficiency, scalability and no overhead on managing servers.

AWS is the leading cloud provider, covering roughly 31% of the global market as of 2026. Part of the reason it is so widely used is because it was the first to market, so it had the most amount of time out of all competitors to refine their services and offerings, which is half of the reason it was chosen. The other half was because the author of this thesis project is a certified Solutions Architect Professional on AWS and had a bias to pick this provider.

4.2.1 Serverless computing

Contrary to the first impression ”serverless” leaves, it doesn't actually mean that you do not have servers. You do need servers to run anything but the term ”serverless” comes from the fact that you are not the one managing them.

So not only are you in the cloud where there is less overhead than actually having a physical data center in your home, but you are not even spinning up virtual machines either, you rely on AWS to manage the infrastructure completely for you. Which is great from many point of views, but can also be a bad choice, depending on the situation.

Advantages of going serverless

Going serverless means you never worry about scaling your services. They can (in theory) handle any amount of traffic spikes without you needing to intervene. It also means you do not need to worry about the availability of the service.

For example, on AWS, when you deploy a Lambda function you do not need to deploy it in multiple Availability Zones (AZs), as you would have to do for EC2 (Elastic Compute Cloud) instances.

You also do not pay for something if you are not using it. If you would boot up an EC2 instance then you are billed by the second (or hour) whether the instance receives traffic or not. In the case of Lambda however, if it only receives traffic for 5 minutes a day then you are paying for those 5 minutes. Not one extra cent.

Disadvantages of going serverless

There are also disadvantages of going serverless, although not as many as there are advantages, they are worth considering.

Although it is cheaper to run serverless when you do not have so much traffic, if you are experiencing constant and high traffic it can actually become more expensive. And in those situations it's usually expensive to re-architect your application too, so you either stick with it or bite the bullet and re-architect.

Cold starts. These happen when our service was not used for some time, hence it is not warmed up and already running and it needs to boot up. It typically takes between milliseconds to a few seconds, depending on the size of the package that is being loaded.

Another one is having less control of your infrastructure, although for most use cases it is more of an advantage than a disadvantage. It's only in very specific cases where it is going to hold you back.

4.3 The architecture

So now that the basics were covered, let's dive into the actual deployment of Doclane, what services were used and why.

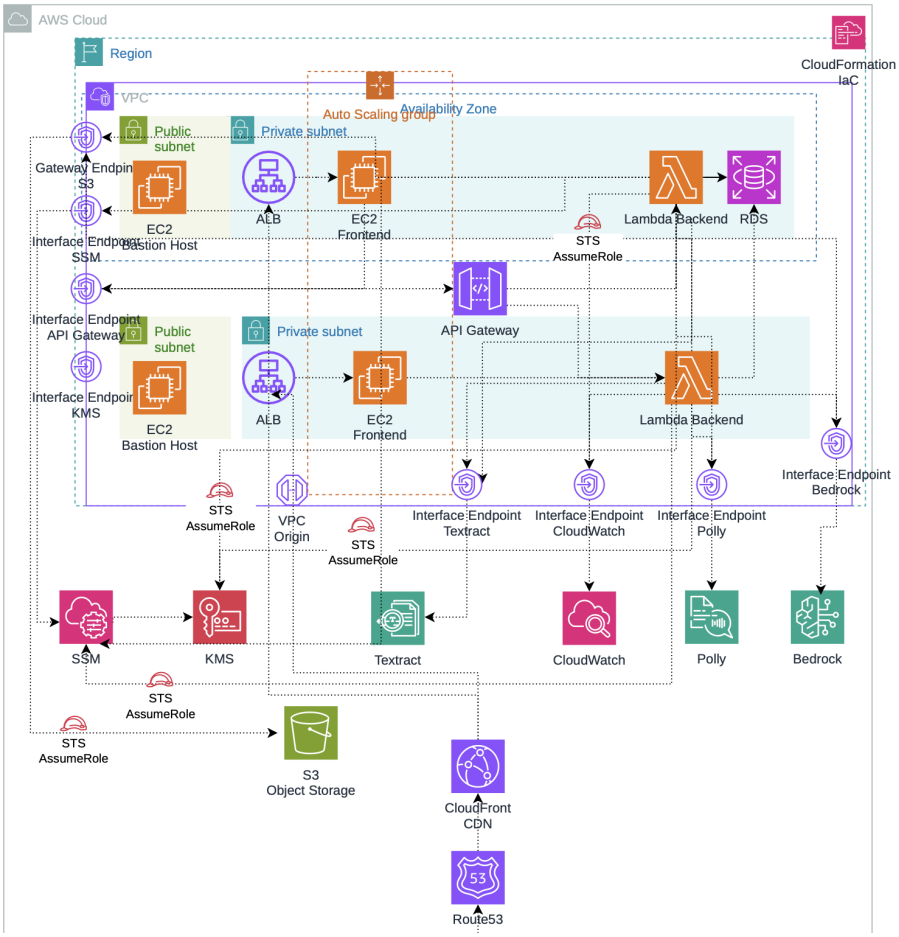

Figure 4.1: Architecture of the AWS deployment.

4.3.1 Networking

The networking part of Doclane consists of a VPC (Virtual Private Cloud) in the eu-west-1 Region on AWS. This VPC consists of multiple subnets, including two private subnets and two public subnets. The private subnets host the database and backend layers, so that no public internet access can be established to them.

The public subnets host the ALB (Application Load Balancer) instances, which balance the traffic to our web servers. They are also used to host an EC2 (Elastic Compute Cloud) instance that is used as a bastion host to have remote SSH access into the database layer.

It is also worth noting that both the private and public subnets span two AZs (Availability Zones), such that high availability is achieved. In case one data center

from AWS goes down where our application runs, then it will still continue to run as if nothing has happened.

An internet gateway was provisioned and added to the route table of the public subnet, such that internet access is possible to and from it. Private subnets are associated with a corresponding route table that only has local routes and the public subnet is associated with a route table that contains a route to the internet. So the thing that makes a subnet public or private is whether or not the associated route table has a route to the internet gateway or not.

Security

When it comes to network security, the architecture leverages Security Groups for securing traffic at the Lambda level and at the RDS level. It also leverages NACLs (Network Access Control Lists) for securing traffic at the subnet level, WAF (Web Application Firewall) for securing layer 7 (application layer) traffic and Shield for layer 3 (network layer) and layer 4 (transport layer) attacks.

A Security Group in AWS is a stateful firewall that operates at a single resource level. This means it can operate at Lambda level, EC2 level, RDS level, but not at the subnet level. It consists of inbound / outbound rules, inbound rules meaning what kind of traffic is allowed (on which port and CIDR range), while outbound rules control what traffic is allowed to go out of our resource that sits behind this firewall.

What makes Security Groups stateful is that if some traffic is allowed inbound into our resource (by the inbound rules), then the corresponding outbound traffic is allowed out from our resource regardless of whether we explicitly allow it or not in the outbound rules. Security Groups work with ”allow” type of rules. Everything is denied by default.

A NACL (Network Access Control List) in AWS is a stateless firewall that operates at the subnet level. It cannot be attached to resources such as Lambda, EC2, RDS, but rather to the subnets that contain them. It consists of inbound / outbound rules, inbound rules meaning what kind of traffic is allowed in, while outbound rules control what traffic is allowed to go outside the subnet.

What makes NACLs stateless is that if some traffic is allowed inbound into our subnet (by the inbound rules), that does not make the corresponding traffic automatically allowed to go out of the subnet. Outbound traffic must be explicitly allowed for it to be able to go outside, regardless of where it originates from.

WAF (Web Application Firewall) in AWS is a service that is basically a virtual firewall that can be applied to some resources. It contains features such as filtering out spam traffic, bot traffic, enforces rate limiting on your resources, and protects your service from common application layer exploits such as cross-site-scripting,

SQL injections and more. In our case, it is applied to the CloudFront distribution, but more on that later.

Shield in AWS is a service that handles network and transport layer exploits such as DDoS (Distributed Denial of Service) attacks. It comes by default with the CloudFront distribution we are using. More on that later.

The entire communication within the application, except for the traffic that comes from clients, is done privately, over AWS network. It never reaches the public internet. Lambda talking to RDS (Relational Database Service) is private, Lambda talking to S3 is private, Lambda putting logs in CloudWatch is private, Lambda getting configuration parameters from SSM (Systems Manager) Parameter Store is private.

This is achieved with the use of VPC Endpoints, specifically the Interface Endpoint type. The only one place a Gateway Endpoint is used over an Interface Endpoint is when Lambda talks to S3. Interface endpoints are part of AWS Private Link, and they leverage an ENI (Elastic Network Interface) that is assigned a private IP and serves as the main entry point for a specific service.

Unline Interface Endpoints, Gateway ones are not part of Private Link and they do not have a private IP, they work more like an Internet Gateway redirecting traffic to a certain prefix list of AWS, including S3 and DynamoDB, without going through public internet.

4.3.2 Database layer

RDS (Relational Database Service) was the preferred choice for the database. As a relational database was needed, the natural choice was AWS' managed service for this, hence why it was preferred.

Other choices such as Aurora were avoided since they are more expensive and it's typically for big companies that require ultra high performance. RDS was the appropiate choice for the allocated budget in this project.

RDS is built in such a way that AWS handles all the operating system patching of the server that is running your database, and you also benefit from automated, scheduled backups out of the box, and other advantages.

The engine is PostgreSQL, chosen for no particular reason rather than it being popular and fast. RDS supports PostgreSQL natively as well.

Security

The RDS database is protected by a Security Group that only allows inbound traffic from Lambda's Security Group, which makes it safe as any traffic not originating from Lambda will be dropped. Granting inbound access only to a certain Security Group is a best practice on AWS, as this is as granular as permissions can get.

It is deployed in a private subnet so no public IP was assigned to it, and no internet access is possible to it. The only way to access it is through a bastion host that was set up in the public subnet.

4.3.3 Backend layer

Lambda was the preferred choice for the backend. A special technique is being used here, called "Lambda in a VPC (Virtual Private Cloud)", but more on this shortly. The reason Lambda was chosen is for the fact that is a serverless compute service, making the development of the application cheap as nothing is running constantly, rather it only runs when it needs to run (i.e. when requests from users reach our servers).

Lambda is intended for short workloads, which is exactly what is needed for Doclane. In order to have an idea for Lambda's intended running time, the default that AWS sets for it is 3 seconds, and the maximum one can go for is 15 minutes.

In our case, a typical request might last 1 second, or slightly more depending on if it is a cold start. A cold start (as covered in the disadvantages of serverless architectures section) happens when our serverless service (such as Lambda) was not used for some time, hence it is not warmed up and ready to serve requests. Cold starts can take from milliseconds to seconds, depending on the size of the package that needs to be loaded.

Lambda in a VPC

By default, Lambda operates in an AWS-owned VPC. This means that it cannot access your private resources, and it natively has access to the internet.

This is great for most use cases you encounter, but in this case it was needed for our Lambda to have access to the RDS database, hence why it was deployed in the Doclane VPC. That made it so our Lambda required to be able to create ENIs (Elastic Network Interfaces) for itself so that it had a private IP and could communicate with other services in the Doclane VPC. These permissions were granting through the usage of IAM (Identity Access and Management) which is another service in AWS, this will be covered later.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ec2:CreateNetworkInterface",
                "ec2:DescribeNetworkInterfaces",
                "ec2:DeleteNetworkInterface",
                "ec2:AssignPrivateIpAddresses",
                "ec2:UnassignPrivateIpAddresses"
            ],
            "Resource": "*"
        }
    ]
}
```

Figure 4.2: The granular permissions that allow Lambda to manage its ENIs

An ENI (Elastic Network Interface) is AWS' abstraction of a network card. It can be attached to resources in order to give them properties such as private IPs, public IPs, MAC addresses, Security Groups and more. What AWS did was basically decouple the networking logic from the resource itself, which is a brilliant move.

4.3.4 Logging and monitoring

In order to ensure that what happens in the application is monitored, logging has been set up in the backend. All the logs that the backend generates are being sent to CloudWatch by the Lambda function (this is why it needs permissions to push logs to CloudWatch).

In case some errors occurs in the app, the debug can happen quickly, since logs can be filtered by type for example, out of many other options.

4.3.5 Identity Access and Management

Throughout the entire deployment, the principle of ”least privilege” was applied, as a best practice not only on AWS but in general. In AWS, security is managed in multiple ways, but the main one is the IAM service, which is used in every single place where something exists on AWS.

For example, the Lambda function in the backend assumes an IAM role attached to it which allows it to publish logs to CloudWatch (more on this later), access the documents S3 bucket, and also create ENIs for itself.

Role assumption in IAM is the concept of generating temporary credentials (in this case valid for one hour) in place of generating permanent credentials, which if compromised might give intruders unwanted access to your account. During a role assumption, the one who assumes the role drops all his permissions and receives only the permissions of the assumed role, for the certain period of time for which it assumes it. Lambda in this case, temporarily assumes the role to be able to see

objects in the S3, upload, and delete them, post logs to CloudWatch or create ENIs for itself.

The aforementioned role that allows access to the S3 bucket, ENIs and CloudWatch is also taken as far as to only allow Lambda to use it. Nothing else can assume this role, not other users or other services. The Trust Policy of it explicitly states only Lambda can assume it. The trust policy looks like this:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "Service": "lambda.amazonaws.com"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

Figure 4.3: Trust Policy of the IAM Role used by Lambda

Since no permanent credentials are ever used in the unfortunate case that someone manages to find a way to steal temporary credentials, the damage they can cause is minimal. In the case of permanent credentials, you have to periodically rotate them in case they were compromised, and check when they were used and what actions were performed using them.

The Lambda backend is given granular, least privilege permissions. It is only able to assume the role that can upload files to S3, create ENIs for itself and publish logs to CloudWatch. Apart from that, it is denied any other action.

4.4 Infrastructure as Code - CloudFormation

Whenever deploying applications in the Cloud, you can manually click in the AWS console for example, and provision everything you need. It's straight forward, intuitive and it provides a friendly experience to the user who sees the components of his applications beautifully put in a visual interface.

Although this is convenient, it is a manual process. And manual processes are prone to errors. It becomes easy to forget what change in configuration you made last week, more simply put, it's a mess.

That's where the concept of IaC (Infrastructure as Code) comes into play. This involves specifying your infrastructure in a declarative manner, and having everything defined as code, it becomes much easier to maintain and have control over.

This is definetly not an easy concept, and one must have a solid grasp on what needs to be deployed and how, up to the finest details, otherwise it just won't work. It is not a technique you use for your first deployment, but rather when you have built some experience and understand what you are doing.

CloudFormation is the chosen tool for the project. It is provided by AWS hence there is a tight integration between the entire ecosystem of AWS and this tool.

```
147
148     PublicSubnet2Association:
149         Type: AWS::EC2::SubnetRouteTableAssociation
150         Properties:
151             RouteTableId: !Ref PublicRouteTable
152             SubnetId: !Ref PublicSubnet2
153
154     GatewayEndpointS3:
155         Type: AWS::EC2::VPCEndpoint
156         Properties:
157             ServiceName: !Sub com.amazonaws.${AWS::Region}.s3
158             RouteTableIds:
159                 - !Ref PrivateRouteTable
160             VpcEndpointType: Gateway
161             VpcId: !Ref VPC
162             Tags:
163                 - Key: Name
164                   Value: doclane-gateway-endpoint-s3
165                 - Key: Description
166                   Value: Gateway endpoint is needed for Lambda to access S3. Lambda backend has no internet access.
167
168     InterfaceEndpointCloudWatch:
169         Type: AWS::EC2::VPCEndpoint
170         Properties:
171             ServiceName: !Sub com.amazonaws.${AWS::Region}.logs
172             VpcEndpointType: Interface
173             SubnetIds:
174                 - !Ref PrivateSubnet1
175                 - !Ref PrivateSubnet2
176             SecurityGroupIds:
177                 - !Ref EndpointSG
178             PrivateDnsEnabled: true
```

Figure 4.4: Sample of CloudFormation code for the VPC.

4.5 Machine Learning

This section goes into detail on the AWS services leveraged to build the machine learning assisted features aforementioned in the implementation chapter.

4.5.1 Textract

Textract is a fully managed serverless service offered by AWS that extracts text from documents. API calls are made to Textract from Doclane in order to extract text from documents.

4.5.2 Polly

Polly is a fully managed serverless service offered by AWS that generates speech from text. API calls are made to Polly to generate speech from selected documents.

4.5.3 Bedrock

Bedrock is a fully managed serverless service offered by AWS that gives you access to different LLMs that you can prompt. In our case we use the Nova Lite model from

Amazon to interpret whether a document provided by an user is actually what it is required by the request.

4.6 Web servers

The frontend part of the app is hosted on EC2 (Elastic Compute Cloud) instances that are part of an ASG (Auto Scaling Group) in private subnets, load balanced by an ALB (Application Load Balancer) which is also deployed in the private subnets, frontend by a CloudFront distribution that actually servers the user traffic.

This tight security setup ensures no other access than through CloudFront is permitted to go into the ALB, and therefore on our web servers. This helps in multiple ways, some of them being: CloudFront is natively integrated with WAF, making it essentially a firewall for the traffic that wants to go through to the ALB, and not only that but it ensures that the application always serves cached content if possible, resorting to fetching data from the server only in case it's not in the cache.

4.6.1 EC2

Elastic Compute Cloud is a service offering from AWS that gives you servers that you manage yourself. All operating system patches, security patches etc. are handled by you. The only thing that AWS gives you is compute power, memory (RAM) and networking capabilities. EC2 gives you full control over everything, even over stuff that would normally be managed by AWS in many different scenarios.

Amazon Machine Image

AMIs are basically templates for your servers. In Doclane's case, an image has been preconfigured with the cloned frontend code, all dependencies installed and ready to spin up the server. If you were to do things the old fashioned way, every time you started a new server you would install dependencies and clone the repository at the very least, which would result in longer boot times. Having a preconfigured (or as the AWS community calls it: ”Golden AMI”) is the way to avoid long startup times.

There is a significant reduction in time needed for the servers to spin up in case high traffic is happening, since the servers only need to boot up and then start the web application. No other commands need to be run.

Auto Scaling Groups

Manually spinning up servers during peak traffic would be cumbersome. Hence, Auto Scaling Groups are here to ensure that the application scales on its own, always being able to handle sudden spikes. Think of Auto Scaling Groups as an orchestrator for EC2 Instances. It ensures that at least a minimum and at most a maximum of instances are running. As a developer, you set those limits.

In Doclane's case, CPU is being tracked to be at most at an average of 70% utilization. If it ever goes higher than that, additional servers are being launched to offload the traffic.

Auto Scaling Groups don't just manage random EC2 Instances, they manage what's called a Target Group. A target group is what actually groups EC2 Instances together in one bundle. Think of it as a "tag" for those instances, making the Auto Scaling Group know what it is managing.

Load Balancing

In order to ensure scalability and high availability, the servers that are inside of the Auto Scaling Group are being fronted by an Application Load Balancer.

An Application Load Balancer in AWS is in essence a reverse proxy, splitting requests from clients among the targets it manages. This ensures uniform distribution of requests on the web servers, not overloading one and underloading another.

The routing algorithm that is being used is Round Robin, what this means as simple as possible is that it sends requests one by one to each target, in order. So after sending a request to the first server, the next request will go to the second one.

Another popular routing algorithm is the Least Outstanding Requests, which ensures that it sends requests to the servers with the least ongoing requests. However Round Robin is usually suited for short lived tasks.

4.6.2 CloudFront

CloudFront is a CDN (Content Delivery Network) offered by AWS that leverages global locations (called "Edge Locations") to cache content close to the users. Latency is significantly reduced since commonly accessed content lives close to the users, hence improving the overall user experience of the application.

You have full control over caching behaviour, including the ability to invalidate the cache at any moment, making it a powerful tool. CloudFront is massively beneficial to reduce traffic on your architecture, acting as a buffer between users and your servers.

4.7 Continuous Integration / Continuous Deployment

Modern practices of deployment have been adopted, such that the deployment pipeline of the application is fully automated. Once changes are pushed to the main branch, everything that needs changing in the AWS environment automatically changes.

This reduces the risks of human error, enables developers to release tens of patches per hour if needed, without having to worry about the overhead of manual deployments.

CI/CD has been done with the help of GitHub Actions. GitHub Actions are convenient to use when your repository is already on GitHub, hence why they were chosen.

Without going into too much detail, it is worth knowing how secure this entire process is. Let's take the backend deployment as an example.

Updates happen leveraging the least-privilege principle, without any permanent credentials ever existing or being stored anywhere. The GitHub Action worker generates and ID token using GitHub's OIDC (OpenID Connect is a popular authentication protocol, built on top of OAuth). GitHub then signs this token with their private key, and this signed token is then passed to AWS to prove the identity of the runner.

Now AWS won't just see a token signed by GitHub and accept it, that would be catastrophic. What happens instead is that in that token signed by GitHub there is useful information about the runner such as which repository it's running in, who owns it, on what branch it's running, and so on. This is trustable information since nobody can access the repository and push without being allowed by the owner of the repository.

In AWS, an OIDC provider is created in IAM, trusting GitHub and allowing Role Assumptions for the worker as long as it proves it's from exactly the main branch of the Doclane repository. The security is so tight that even changing the branch you are pushing to will deny the worker any access to AWS.

Role Assumption in AWS is a concept that allows generating temporary credentials to perform actions in an AWS account, leveraging the STS (Secure Token Service) from AWS. Credentials are short-lived, for example in Doclane the shortest duration was chosen, that being one hour. Once the credentials expire they cannot be used anymore, and even if they somehow get leaked the damage would be minimal.

On top of that, not only is it using temporary credentials but the access it gets with those temporary credentials is strictly limited to updating one specific Lambda function in the account. So even if they did ever get leaked, nothing would be al-

lowed other that simply updating Doclane's backend lambda code. No other permissions than that are given to the temporary authenticated worker.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "Federated": "arn:aws:iam::659775407830:oidc-provider/token.actions.githubusercontent.com"
            },
            "Action": "sts:AssumeRoleWithWebIdentity",
            "Condition": {
                "StringEquals": {
                    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
                    "token.actions.githubusercontent.com:sub": "repo:Robert076/doclane:ref:refs/heads/main"
                }
            }
        }
    ]
}
```


Figure 4.5: The least-privilege principle shown in the assumed role by the worker.

Chapter 5

User workflow

This chapter describes how the system is used by end users and explains typical workflows for common scenarios in daily use.

5.1 Registration and login

The first step for using the system is creating an account. The user must register by providing basic information such as name, email, and password. After registration, the account is created and can be used to access the system.

After registration, the user can log in using their email and password. Once logged in, the system creates a session for the user and allows access to the available features based on their role.

5.2 Submitting a request

After logging in, the user needs to complete their profile by adding personal information such as address and phone number.

After the profile is completed, the user can submit a request. The user can choose a request template from the available list and fill in the required information. Depending on the template, the user may also need to upload supporting documents.

Once the request is submitted, it is sent to the relevant department in the public institution. Employees review the submitted information and check whether all required documents are correct and complete.

5.3 Review and feedback process

During the review process, each submitted document is checked individually by the department employee. Each document can be either approved or rejected sepa-

rately.

If a document is correct and meets the requirements, it is marked as approved. If a document is incorrect, missing information, or does not meet the requirements, it is rejected.

When a document is rejected, the employee can add a reason for the rejection. This helps the user understand what needs to be corrected or resubmitted. The user receives this feedback directly in the system.

After receiving feedback, the user can update the request by uploading corrected documents. The request is then reviewed again until all documents are approved.

When all documents in a request are approved, the request is marked as completed and the user is notified. The user can then collect the final document from the institution if needed.

5.4 Administrator workflow

Administrators are responsible for managing request templates in the system.

An administrator can create a new template for a specific type of request. Each template defines what documents are required and what information the user must provide. The administrator can also add example files for each required document to help users understand what is expected.

In addition, templates can be organized using tags. Tags help group similar templates together and make it easier to search and manage them inside the system.

5.5 Department workflow

Each request belongs to a specific department inside the institution. Employees from that department are responsible for handling incoming requests.

When a new request is submitted, it becomes available to department members. A department member can then claim a request, which means they become responsible for reviewing and processing it.

After claiming a request, other employees can see that it is already assigned and avoid duplicate work. The assigned employee continues the review process until the request is completed.

This system ensures that each request has a clear owner and improves organization inside the department.

5.6 AI-assisted document processing

The system includes AI-based tools to help with document processing. These tools are used to assist employees during the review of uploaded documents, especially when the document is unclear or hard to read.

One of the main use cases is when a document is blurry or of low quality. In this case, an AI service can analyse the image and estimate whether the document matches the expected type defined in the request template. This helps the employee decide faster if the document is valid or needs to be rejected.

Another feature is text extraction from images. If a document is scanned or uploaded as a photo, an OCR (Optical Character Recognition) service can extract the text content from it. This makes it easier to read and search through documents without manually inspecting every file.

In some cases, a text-to-speech (TTS) service can also be used to convert document content into audio. This can help users or employees who prefer listening instead of reading, or when reviewing long documents.

These AI tools do not make final decisions. They only provide suggestions or extracted information, while the final approval or rejection is always done by the assigned department employee.

Chapter 6

System limitations and discussion

This chapter discusses the limitations of the current system and explains some design decisions made during development. While the system covers the main workflow of document processing in public institutions, there are still areas that can be improved.

6.1 System scope

Doclane is designed as a workflow system for handling document-based requests between users and public institutions. It does not change legal procedures or replace existing administrative rules. Final decisions are still made by authorized employees within the institution.

The system focuses only on digital support of existing processes. It does not automate decision-making in a legal sense, but instead helps organize and speed up document handling.

6.2 Dependence on external services

The system uses external services for file storage and AI-based processing. This includes cloud storage for uploaded files and third-party services for tasks such as text extraction and document analysis.

This introduces a dependency on external systems. If one of these services is unavailable, some features of the system may be limited. However, the core functionality of the system remains usable.

6.3 AI limitations

AI tools are used to assist with document processing, but they do not make final decisions. They can extract text from images or help identify document types, but the final validation is always done by the assigned employee.

In some cases, AI results may not be fully accurate. Because of this, AI is only used as a support tool and not as a replacement for human review.

6.4 User adoption challenges

One possible limitation of the system is user adoption. Some users may not be familiar with digital systems or may prefer traditional paper-based processes.

In addition, employees in public institutions may require time to adapt to the new workflow. Training and experience are important for the system to be used effectively.

6.5 Scalability and future use

The system is designed to be scalable and support multiple institutions. However, large-scale adoption would require additional configuration, integration with national systems, and stronger legal frameworks.

At the current stage, the system serves as a prototype that demonstrates how document workflows can be improved using digital tools.

6.6 Summary

The system successfully covers the main workflow of document submission and processing, but it still has limitations related to external dependencies, AI accuracy, and user adoption. These limitations define the areas for future improvement and further development of the system.

Chapter 7

Conclusion

This thesis presented Doclane, a web-based system for managing document-based workflows between citizens and public institutions. The system was designed to reduce the need for physical visits and paper-based processes by moving the workflow into a digital environment.

The main goal of the system is to make the submission and processing of documents simpler and more structured. Users can submit requests online, upload required documents, and track the status of their application. Employees in public institutions can review submissions, approve or reject documents, and provide feedback when needed.

The system also introduces role-based workflows where each request is handled by a responsible employee. This helps keep the process organized and avoids confusion in document handling.

In addition, AI-based tools were integrated to assist in document processing. These tools can help extract text from images, check document quality, and provide additional support during the review process. However, final decisions are always made by the assigned employee.

The system was implemented using a Go backend and a Next.js frontend. Cloud services were used for file storage and deployment, making the system scalable and accessible.

Although the system covers the full workflow between users and institutions, there are still areas that can be improved in the future.

7.1 Future improvements

One important improvement is the introduction of two-factor authentication (2FA). This would increase the security of user accounts and reduce the risk of unauthorized access.

Another key improvement is the integration of eIDAS-compliant digital signing. This would allow users to verify their identity in a stronger way and reduce or fully remove the need for physical presence at public institutions.

A third improvement is the introduction of a clear request timeline. This would show all steps of a request in chronological order, including submission, review, feedback, and final approval. This would improve transparency for users and make the process easier to understand.

These improvements would make the system more secure, more transparent, and closer to a fully digital public administration workflow.

Bibliography

[AYMJ19] Azlina Ab Aziz, Zawiyah Mohammad Yusof, Umi Asma Mokhtar, and Dian Indrayani Jambari. The implementation guidelines of digital document management system for malaysia public sector: Expert review. *Journal of Physics: Conference Series*, 1333:072034, 2019.

[dms12] System integration of document management systems in public sector institutions. *International Journal of Advances in Engineering & Technology*, 5(1):194–205, 2012. Study on integration of document management systems with other information systems in public sector organizations.

[KZM19] S V Koptyakova, E G Zinovyeva, and T V Maiorova. Development and deployment of automated electronic document management system in municipal units. *Journal of Physics: Conference Series*, 1333, 2019.